Student Dashboard

Next-Gen Learning Management Dashboard

Technology Stack:	Next.js 16, React 19, TypeScript, Tailwind CSS v4, Supabase, Framer Motion
Architecture:	App Router with Server/Client Component split
Database:	Supabase PostgreSQL (5 tables with RLS)
UI Framework:	shadcn/ui + Base UI React + Lucide Icons
Live URL:	https://dashboard-cyan-alpha-13.vercel.app
GitHub Repo:	https://github.com/aravinds90g/student-dashboard
Date:	June 06, 2026

1. Project Overview

The Student Dashboard is a modern, interactive learning management system designed to provide students with a comprehensive view of their educational progress. Built with the latest web technologies, it features a dark-themed, animated interface with real-time data from a Supabase PostgreSQL database.

Key Features

- Interactive bento-grid dashboard with animated tiles and staggered entrance animations
- Real-time course progress tracking with visual progress bars and module breakdowns
- Assignment management with status tracking (pending, submitted, graded, overdue)
- Certificate showcase with credential IDs, grades, and time-spent tracking
- Weekly activity charts with bar graph visualization and compute efficiency metrics
- Streak tracking with day-of-week indicators and motivational elements
- Upcoming class schedule with time, duration, and color-coded categories
- Fully responsive layout: desktop sidebar, tablet collapsed nav, mobile bottom bar
- Dark theme with orange accent (#FF6A00) and animated mouse-responsive border glow

2. Technical Architecture

2.1 Server/Client Component Split

The application follows Next.js App Router conventions with a deliberate separation between server and client components:

Server Component (src/app/page.tsx):

Fetches all dashboard data from 5 Supabase tables in a single Promise.all call. This keeps data fetching on the server, reduces client bundle size, and eliminates waterfall requests. On error, it renders an ErrorFallback component instead of crashing.

Client Component (src/components/HomeClient.tsx):

Receives pre-fetched dashboardData as props and manages client-side tab state. Serves as the orchestration layer, conditionally rendering the dashboard grid, courses, assignments, certificates, or settings page based on active tab.

Presentational Components:

Individual tiles (bento-grid-demo-2.tsx, CourseTile.tsx, ActivityTile.tsx) are pure presentational client components with staggered Framer Motion entrance animations triggered by parent container variants.

2.2 Data Flow

Data flows unidirectionally from Supabase PostgreSQL through the server component to client components via props:

```
Supabase (PostgreSQL)
  -> Server Component: parallel fetch of 5 tables
    -> DashboardData type as props
      -> HomeClient (tab router)
        -> BentoGridSecondDemo / CoursesPage / etc.
```

2.3 Responsive Design

The layout adapts across three breakpoints using Tailwind CSS responsive utilities:

Breakpoint	Sidebar	Grid Columns
>1024px (desktop)	Full sidebar with icons	3 columns
768-1024px (tablet)	Collapsed icons only	2 columns
<768px (mobile)	Bottom navigation bar	1 column

3. Technology Stack

Next.js 16	App Router framework with React Server Components and Turbopack
React 19	Latest stable React with Server Components and Actions
TypeScript 5	Type-safe development with strict mode and discriminated unions
Tailwind CSS v4	Utility-first CSS with CSS-first configuration and custom dark mode variables
Framer Motion 12	Spring-physics and layout animations, staggered children, gesture interactions
Supabase	PostgreSQL database with Row Level Security for real-time dashboard data
shadcn/ui 4	Reusable UI primitives built on Base UI React with CVA variants
Lucide React	Lightweight tree-shakable icon library with stroke-based design

4. Project Structure

The project follows a clean src-based directory structure:

- public/ -- Static images (course backgrounds, SVGs)
- scripts/seed.mjs -- Database seed script (creates 5 tables + sample data)
- src/app/ -- Next.js App Router pages (layout.tsx, page.tsx, globals.css)
- src/components/ -- All React components organized by domain (dashboard, courses, assignments, certificates, settings, ui)
- src/components/BorderGlow.tsx -- Animated mouse-responsive border glow effect
- src/components/ActivityTile.tsx -- Weekly activity bar chart with compute metrics
- src/components/HomeClient.tsx -- Client-side tab router and page orchestrator
- src/components/Sidebar.tsx -- Desktop sidebar + mobile bottom navigation
- src/data/ -- Static mock data for courses, assignments, and certificates
- src/lib/ -- Supabase client factory and cn() utility
- src/types/ -- TypeScript interfaces for all data models

5. Database Schema

The application uses 5 PostgreSQL tables, all created automatically by the seed script with Row Level Security enabled and public SELECT policies for anonymous access.

courses

id (uuid PK), title (text), progress (int), icon_name (text), created_at (timestamptz)

user_stats

Single row: name, ahead_percent, learning_hours, completed_courses, certificates, streak_days, etc.

activities

id (uuid PK), day (text), hours (int), created_at -- 7 rows, one per day of week

achievements

id (uuid PK), title (text), icon (text) -- e.g. '7-Day Streak', 'Top 10% Learner'

upcoming_classes

id (uuid PK), title (text), time (text), duration (text), color (text)

All tables use UUID primary keys with gen_random_uuid() defaults. RLS policies are created with IF NOT EXISTS guards for idempotent seeding.

6. Component & Page Overview

Dashboard (Home)

Bento-grid layout with hero section (greeting, stats, progress), streak tracker with day indicators, mini course tiles with animated progress bars, achievements, weekly activity bar chart, and upcoming classes grid. All tiles use staggered Framer Motion entrance animations.

Courses

Overview with 4 stat cards (total, in-progress, completed, hours). Course cards show progress, difficulty badges, instructor. Detail view shows full progress bar, module-by-module breakdown, and deadline date.

Assignments

Overview with stat cards by status. Cards show course, description, difficulty, estimated hours, days remaining. Detail view has full description, score visualization (if graded), overdue warning, and submit button.

Certificates

Overview with stat cards. Certificate cards show grade badge, instructor, issue date, credential ID. Detail view has grade bar, dates, and time spent.

Settings

Profile section with avatar. Notification toggles (email digest, reminders, course updates, reports). Account section with plan, password, 2FA, sessions. Danger zone with account deletion.

Sidebar

Desktop: vertical icon bar with tooltip labels, logo, logout, user avatar with online indicator. Active tab has orange glow via layoutId. Mobile: fixed bottom nav bar with 6 tabs.

BorderGlow

Reusable animated border glow component responding to mouse proximity. Uses CSS custom properties and conic-gradient masks for a cursor-following highlight effect with configurable colors and intensity.

7. Setup & Running Instructions

Prerequisites

Node.js 20+, npm, and a Supabase project (free tier).

Installation Steps

1. Clone the repository and navigate to the dashboard directory
2. Run 'npm install' to install all dependencies
3. Copy .env.example to .env.local and fill in Supabase URL and anon key
4. Set DB_PASSWORD in .env.local with your Supabase database password
5. Run 'npm run seed' to create tables and populate sample data
6. Run 'npm run dev' to start the development server
7. Open http://localhost:3000 in a browser

Environment Variables

NEXT_PUBLIC_SUPABASE_URL: Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY: Public anonymous API key
DB_PASSWORD: Supabase database password (used by seed script)
DATABASE_URL: Full connection string (alternative to DB_PASSWORD)

Production Build

Run 'npm run build' followed by 'npm start'. Ensure all environment variables are set in the production environment.

8. Design Decisions & Trade-offs

1. Server-side data fetching

All Supabase queries run in a single server component Promise.all, avoiding client-side data fetching overhead and waterfall requests. Trade-off: data is fetched on every navigation (no client cache) -- a SWR/React Query layer could be added for caching.

2. Framer Motion for animations

Chosen over CSS animations for spring-physics support, layoutId transitions, and staggered children. All animations use transform/opacity only to avoid layout repaints. Trade-off: ~30KB client bundle addition.

3. Custom BorderGlow component

Built with CSS custom properties and conic-gradient masks instead of a library. Provides mouse-responsive edge glow with zero JS animation overhead during idle. Trade-off: CSS complexity and mask-composite browser support.

4. Dark mode only

Deep near-black backgrounds (#000000, #111111) with orange accent for a premium, futuristic feel. Trade-off: no light mode toggle -- adding one means duplicating all CSS variable definitions.

5. Static data for sub-pages

While the dashboard fetches from Supabase, Courses/Assignments/Certificates pages use local TypeScript arrays. This avoids DB dependency for secondary pages while keeping the dashboard real-time. Can be migrated to DB queries.

6. Bento grid layout

CSS Grid with col-span utilities creates the magazine-style layout. Trade-off: tile sizes are fixed at breakpoints -- a fully dynamic layout would need a masonry library.

9. Challenges & Solutions

Server/Client Boundary

Passing Framer Motion animated components from server to client required careful separation. The server component fetches data and passes it as props to a client wrapper, ensuring animations don't break hydration.

Supabase SSR Setup

Using @supabase/ssr requires a createServerClient helper with cookie support. The initial implementation used @supabase/supabase-js directly; switching to @supabase/ssr ensured proper cookie-based session handling.

Staggered Animations

Coordinating entrance animations across multiple independently-rendered tiles required using Framer Motion's variants with staggerChildren on the grid parent, rather than manual index-based delays scattered across components.

Responsive Sidebar

Transitioning from a persistent left sidebar on desktop to a bottom navigation bar on mobile required conditional rendering based on viewport width, combined with AnimatePresence for smooth transitions.

CSS Border Glow Performance

The custom border glow uses CSS custom properties and conic-gradient masks. Achieving smooth 60fps mouse tracking while avoiding repaints required isolating all animated properties to compositor-only (opacity and transforms).